# IMAGE - TO TOPIC CLASSIFICATION

Image Classification and Text Generation Using CLIP and FLAN-T5

**PROJECT REPORT**

Submitted in partial fulfillment of the requirements
for the academic project

Submitted by :-

Kashish Bishnoi (2304025)

Vaasav Duharia (2330053)

Abhyudaya Mishra (2330059)

Shubban Sundar (2330268)

**Department of Electronics and Telecommunication Engineering**
**KIIT University, Bhubaneswar**

Academic Year: 2026–2027

# CERTIFICATE

This is to certify that the project report entitled “AI Image Topic Classifier: Image Classification and Text Generation Using CLIP and FLAN-T5” has been prepared as part of the academic project work undertaken by the student of the Department of Electronics and Telecommunication Engineering, KIIT University, Bhubaneswar.

The report presents the problem definition, objectives, system methodology, implementation, testing observations, sample outputs, limitations and future scope of the proposed application.

# ACKNOWLEDGEMENT

I would like to express my sincere gratitude to the faculty members of the Department of Electronics and Telecommunication Engineering, KIIT University, for their guidance and support during the development of this project. Their encouragement and academic inputs helped me understand the integration of computer vision, natural language processing and web technologies.

I am thankful to my project guide for valuable suggestions, constructive feedback and assistance in improving the project structure and documentation. I also acknowledge the developers and maintainers of the open-source tools used in this work, particularly PyTorch, Hugging Face Transformers, Flask and Pillow

Finally, I thank my family and friends for their motivation and support throughout the project. Their encouragement helped me complete the implementation, testing and preparation of this report.

# ABSTRACT

The AI Image Topic Classifier is a web-based application designed to identify the broad subject of an uploaded image and present relevant textual information. The system combines a pretrained vision-language model with an instruction-tuned language model. CLIP (Contrastive Language–Image Pre-training) performs zero-shot image-to-topic classification by comparing the visual representation of an image with text representations of five predefined categories: Farming, Education, Healthcare, Tourism and Transportation. The category with the highest similarity score is selected as the predicted topic.

After classification, FLAN-T5 Base is used to generate five topic-related keywords. A predefined profession-description dictionary supplies a short explanatory paragraph associated with the predicted category. Flask connects the model-processing functions to a browser-based interface, while HTML and CSS provide the upload form and results layout

The application accepts an image, prepares it in RGB format, executes image-text similarity, normalizes the scores and displays the predicted topic and score. It also presents keywords and a profession description. The project illustrates how pretrained models can be integrated into a practical application without training a large model from scratch. It further highlights the distinction between a similarity-based confidence score and measured classification accuracy

Keywords: CLIP, FLAN-T5, image classification, zero-shot learning, vision-language model, natural language generation, Flask, Python

# TABLE OF CONTENTS

1. Introduction

2. Problem Statement and Objectives

3. Proposed Methodology and Processing Pipeline

4. Models and Tools Used

5. System Architecture and Functional Design

6. Implementation and Usability

7. Results and Sample Outputs

8. Testing, Evaluation and Discussion

9. Limitations, Security and Ethical Considerations

10. Conclusion and Future Scope

11. References

Appendix A – Glossary

Appendix B – User Guide

Appendix C – Repository Structure

# LIST OF FIGURES

Figure 5.1 – Overall system architecture (conceptual)

Figure 5.2 – Processing pipeline

Figure 7.1 – Education classification result

Figure 7.2 – Farming classification result

Figure 7.3 – Healthcare classification result

Figure 7.4 – Tourism classification result

Figure 7.5 – Transportation classification result

Figure 7.6 – Keyword and profession-description panels

# LIST OF TABLES

Table 2.1 – Project objectives and measurable outputs

Table 3.1 – Processing stages

Table 4.1 – Software libraries and roles

Table 5.1 – Functional requirements

Table 5.2 – Non-functional requirements

Table 7.1 – Sample category outputs

Table 8.1 – Functional test cases

Table 8.2 – Evaluation metrics

# LIST OF ABBREVIATIONS

| **Abbreviation** | **Meaning** |
|---|---|
| AI | Artificial Intelligence |
| CLIP | Contrastive Language–Image Pre-training |
| NLP | Natural Language Processing |
| ViT | Vision Transformer |
| T5 | Text-to-Text Transfer Transformer |
| UI | User Interface |
| RGB | Red, Green, Blue |
| API | Application Programming Interface |
| CPU | Central Processing Unit |
| GPU | Graphics Processing Unit |
| URL | Uniform Resource Locator |
| HTML | HyperText Markup Language |
| CSS | Cascading Style Sheets |
| HTTP | HyperText Transfer Protocol |
| ML | Machine Learning |

# PROJECT OVERVIEW AND APPLICATION INTERFACE

The developed application is intended to make image classification accessible through a simple browser interface. A user does not need to interact directly with Python functions or model APIs. The homepage provides a file-selection control and a classification button. After an image is submitted, the backend performs the required inference and returns a result page.

The interface separates the user-facing workflow from model execution. This separation improves usability and makes it possible to revise the interface without changing the model logic. The homepage screenshot below is placed near the beginning of the report to introduce the practical system before the detailed technical chapters.

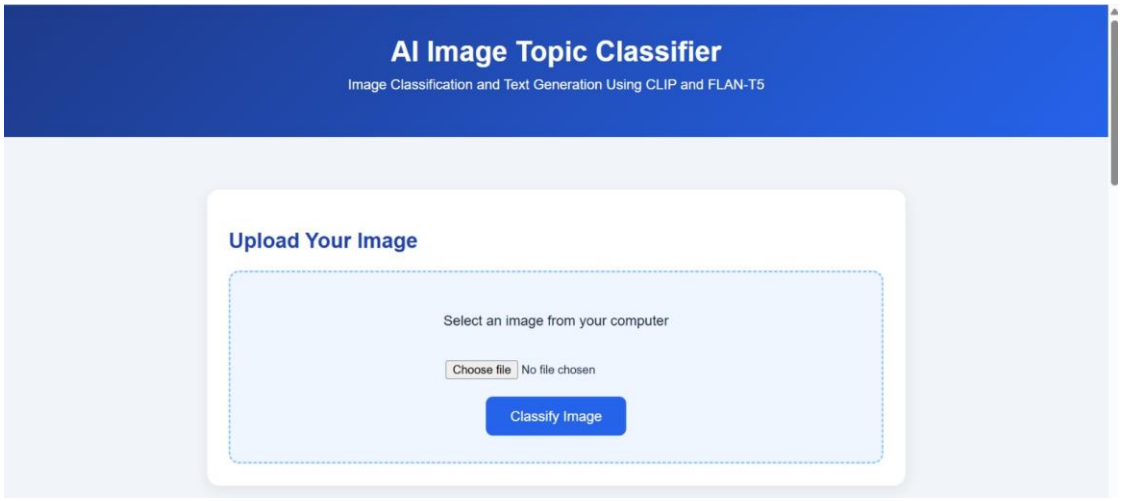



Figure 1.1: Homepage of the AI Image Topic Classifier showing the image upload area and Classify Image action.

Meaning: The screenshot represents the entry point of the application. The user selects an image from the computer and submits it for automated topic classification.

# CHAPTER 1 – INTRODUCTION

Artificial Intelligence (AI) enables computer systems to perform tasks that traditionally require human reasoning, perception or language understanding. Image classification is one of the important applications of AI in which a system assigns a label to an image based on its visual content. Natural language processing (NLP), on the other hand, enables machines to process and generate human-readable language.

## 1.1 Background

Digital images are generated in large quantities across education, agriculture, healthcare, tourism, transport and other domains. Extracting useful information from these images manually can be slow when the number of images is large. Automated image understanding can support indexing, retrieval, content organization and accessibility. Traditional image classifiers are commonly trained on labelled datasets and may be restricted to the classes represented during training

Vision-language models provide a flexible alternative. They learn relationships between visual inputs and text descriptions, allowing an image to be compared with candidate labels written in natural language. CLIP is an example of this approach. Its image and text encoders map the two modalities into a shared representation space where similarity can be calculated

## 1.2 Need for the Project

A practical demonstration is useful for understanding how a pretrained AI model can be connected to a web application. This project addresses that need by creating a small, understandable system with a defined set of categories and a visible output. It also demonstrates that classification alone can be complemented with text generation to make the result more informative for users

## 1.3 Project Scope

The project scope is intentionally limited to five broad topics: Farming, Education, Healthcare, Tourism and Transportation. It is a topic classifier rather than a fine-grained object detector. It does not identify every object, person or activity in an image, and it does not claim to provide professional advice. The output is intended for demonstration and educational use

## 1.4 Organization of the Report

The report begins with the problem statement and objectives, then explains the processing pipeline and models. System architecture and implementation are discussed before the results and evaluation. The final chapters describe limitations, responsible use, future enhancements and references.

# CHAPTER 2 – PROBLEM STATEMENT AND OBJECTIVES

This chapter defines the problem addressed by the application, explains its motivation and establishes the expected deliverables.

## 2.1 Problem Statement

With the rapid growth of artificial intelligence, image processing and natural language processing have become important technologies for understanding and interpreting digital information. The objective of this project is to develop an AI-based Image-to-Topic Classification System that can automatically identify the subject of an uploaded image and generate relevant textual information based on the identified topic

The system focuses on five major categories: Farming, Education, Healthcare, Tourism and Transportation. When a user uploads an image, CLIP identifies the most relevant category. FLAN-T5 then generates five related keywords, while a predefined description provides a short explanation of the corresponding profession. Python and Flask are used to deliver the functionality through a browser

## 2.2 Motivation

Images contain valuable information about activities, professions, locations and sectors. Manual identification and description may be repetitive and time-consuming. A combined image-and-text system can demonstrate a more convenient workflow: visual input is converted into a topic label, and the label is enriched with language output

Another motivation is to study pretrained AI models in a practical setting. Rather than undertaking expensive model training, the project uses publicly available checkpoints and concentrates on inference, integration, interface design and evaluation. This approach is appropriate for a student prototype where computational resources and labelled data may be limited

## 2.3 Objectives

| **Objective** | **Expected output** |
|---|---|
| Classify images into five topics | One predicted category per upload |
| Use pretrained CLIP | Image-text similarity scores |
| Generate topic keywords | Five displayed keywords |
| Provide profession information | Short category description |
| Integrate components with Flask | Browser-based workflow |
| Build a user-friendly interface | Upload and results pages |

## 2.4 Expected Outcomes

- A working web application that accepts image uploads.
- A predicted topic and normalized similarity score.
- Five related keywords for the selected topic.
- A short profession-related paragraph.
- A unified interface for displaying all outputs.

# CHAPTER 3 – PROPOSED METHODOLOGY

The methodology follows a sequential workflow. Each component has a defined input and output so that the complete system can be tested independently and as an integrated application.

## 3.1 Processing Pipeline

User Image Upload → Image Preprocessing → CLIP Model → Topic Classification → FLAN-T5 → Keyword Generation → Profession Description → Display Results

## 3.2 Stage 1: Image Upload

The user selects an image using the file input control. The Flask route receives the submitted file and checks whether a file has been provided. The uploaded image is stored in the application's upload directory for processing and display. A production deployment should additionally validate file type, file size and file content

## 3.3 Stage 2: Image Preprocessing

The image is opened using Pillow (PIL) and converted to RGB. This ensures that grayscale, palette or other supported image modes are normalized before being passed to the model processor. The CLIP processor performs the model-specific transformations, including resizing, cropping, normalization and tensor preparation as required by the checkpoint

## 3.4 Stage 3: CLIP Classification

CLIP compares the uploaded image with textual prompts representing the five categories. The image encoder produces a visual embedding and the text encoder produces embeddings for the candidate descriptions. Similarity is calculated between the image embedding and each text embedding. The highest scoring category is selected as the predicted topic

The implementation applies softmax to the similarity logits to obtain normalized values that sum to one across the candidate classes. The largest normalized value is displayed as a confidence-like score. This value indicates relative preference among the supplied candidate labels; it is not equivalent to empirical accuracy or a calibrated probability of correctness

## 3.5 Stage 4: FLAN-T5 Keyword Generation

The predicted topic is passed to FLAN-T5 through an instruction prompt asking for five relevant keywords. The tokenizer converts the prompt into model inputs, and the encoder-decoder model generates output tokens. Decoding converts those tokens into readable text. The application then separates the generated response into candidate keywords, removes duplicates and applies a fallback list if fewer than five valid entries are available

## 3.6 Stage 5: Profession Description

The current implementation uses a predefined dictionary to map each topic to a short profession description. This provides consistent, predictable text and avoids the variability of generated paragraphs. The paragraph is category-level information and should not be treated as an exhaustive occupational description

## 3.7 Stage 6: Results Display

Flask passes the uploaded image path, predicted topic, score, keywords and description to the results template. HTML structures the content and CSS styles the cards, labels and spacing. The result is rendered in the browser without requiring the user to understand model internals

# CHAPTER 4 – MODELS AND TOOLS USED

The application combines pretrained deep learning models with conventional software components. This chapter describes the role of each technology and the reason for its selection

## 4.1 CLIP: Contrastive Language–Image Pre-training

CLIP was introduced by OpenAI and trained on image-text pairs to learn transferable visual representations. It contains an image encoder and a text encoder. During pretraining, representations of matching image-text pairs are encouraged to be close, while non-matching pairs are separated. This alignment enables the model to compare images with natural-language prompts

The project uses the checkpoint openai/clip-vit-base-patch32. ViT-B/32 denotes a Vision Transformer backbone with a base configuration and patch size of 32 pixels. The checkpoint is used in inference mode and is not fine-tuned on a project-specific dataset

### 4.1.1 Zero-Shot Classification

Zero-shot classification refers to predicting a label without training a dedicated classifier on labelled examples from the target categories. The candidate categories are supplied as text. CLIP's pretrained image-text alignment makes it possible to rank these descriptions by similarity. In this project, the candidate list is fixed to five broad topics

### 4.1.2 Candidate Prompt Design

Prompt wording influences the similarity score. A category can be represented by a simple label or a descriptive sentence such as “a photograph related to farming and agricultural activity”. Consistent prompt structure reduces accidental differences between candidate descriptions. Prompt design should be documented and kept fixed during comparative evaluation

## 4.2 FLAN-T5 Base

FLAN-T5 is an instruction-tuned variant of the T5 text-to-text architecture. T5 treats tasks as text input and text output. FLAN instruction tuning improves the model's ability to follow natural-language task descriptions. The project uses google/flan-t5-base to generate topic-related keywords

The model receives an instruction containing the predicted topic and a request for five concise keywords. Generation parameters such as maximum output length and decoding strategy affect the result. The output is not guaranteed to be perfectly structured; therefore, the application performs basic parsing and fallback handling

## 4.3 Python and PyTorch

Python provides the application logic and integration layer. PyTorch executes tensor operations and neural-network inference. The Transformers library supplies pretrained model classes, tokenizers and processors. Pillow supports image loading and RGB conversion

## 4.4 Flask, HTML and CSS

Flask is a lightweight web framework used to define routes, receive form submissions and render templates. HTML provides the structure of the upload and results pages. CSS controls layout, colors, spacing, typography and responsive behavior. Werkzeug utilities support safe filename handling

## 4.5 Development Environment

| Component | Configuration / role |
|---|---|
| Programming language | Python 3.11.9 |
| IDE | Visual Studio Code |
| Operating system | Windows |
| Environment | Python virtual environment |
| Model source | Hugging Face |
| Browser | Google Chrome |
| Backend | Flask |
| Inference framework | PyTorch |

# CHAPTER 5 – SYSTEM ARCHITECTURE AND FUNCTIONAL DESIGN

The architecture separates the interface, request handling, model inference and result rendering. Such separation makes the application easier to understand, test and maintain

## 5.1 High-Level Architecture

| Layer | Main responsibility |
|---|---|
| Presentation layer | Image upload form, result cards and user feedback |
| Application layer | Flask routes, validation and workflow coordination |
| AI inference layer | CLIP image-text similarity and FLAN-T5 generation |
| Content layer | Category prompts, fallback keywords and descriptions |
| Storage layer | Temporary uploaded images and static assets |

## 5.2 End-to-End Data Flow

1. The user opens the homepage in a browser.
2. The user selects an image and submits the form.
3. Flask receives the file and stores it using a safe filename.
4. The image is opened and converted to RGB.
5. The CLIP processor prepares the image and candidate text prompts.
6. CLIP calculates similarity scores and the application selects the highest-ranked category.
7. The category is supplied to the FLAN-T5 keyword generator.
8. The application retrieves the matching profession description.
9. The results template displays the image, topic, score, keywords and paragraph.

## 5.3 Functional Requirements

| ID | Requirement | Priority |
|---|---|---|
| FR-01 | Allow the user to select an image | High |
| FR-02 | Accept and process the uploaded image | High |
| FR-03 | Classify into five predefined topics | High |
| FR-04 | Display a normalized similarity score | High |
| FR-05 | Generate five related keywords | High |
| FR-06 | Show a profession description | High |
| FR-07 | Display uploaded image on result page | High |
| FR-08 | Provide clear feedback for missing files | Medium |

## 5.4 Non-Functional Requirements

| Requirement | Design consideration |
|---|---|
| Usability | Simple upload and readable result layout |
| Reliability | Handle missing or invalid input gracefully |
| Maintainability | Keep model functions separate from routes |
| Performance | Load models once where practical |
| Security | Validate uploads and use safe filenames |

| | |
|---|---|
| Portability | Document dependencies and setup steps |

## 5.5 Homepage and Interface Meaning

The homepage acts as the entry point and communicates the primary action clearly. The upload panel groups the file selector and classification button. The result page uses visual hierarchy: the uploaded image is shown first, the predicted topic is emphasized, the score is displayed below it, and supporting text is organized into keyword and description cards. This layout helps users interpret the output in a natural sequence

# CHAPTER 6 – IMPLEMENTATION AND USABILITY

Implementation translates the architecture into a working browser-based workflow. The following subsections describe the setup, model loading, processing functions, integration and user operation

## 6.1 Environment Setup

A project folder is created in Visual Studio Code. Python 3.11.9 is installed and a virtual environment is used to isolate dependencies from other Python projects. The environment is activated before package installation or application execution

The main packages include torch, transformers, pillow, flask and sentencepiece. SentencePiece is required by the T5 tokenizer. The model checkpoints may be downloaded on first use, so the first startup can take longer than subsequent runs

## 6.2 Model Loading and Reuse

CLIP and FLAN-T5 are loaded from their Hugging Face identifiers. Model loading is computationally expensive relative to a single inference call; therefore, the recommended design is to initialize the models once when the application starts and reuse them for incoming requests. Reloading the models for every image would increase latency and memory overhead

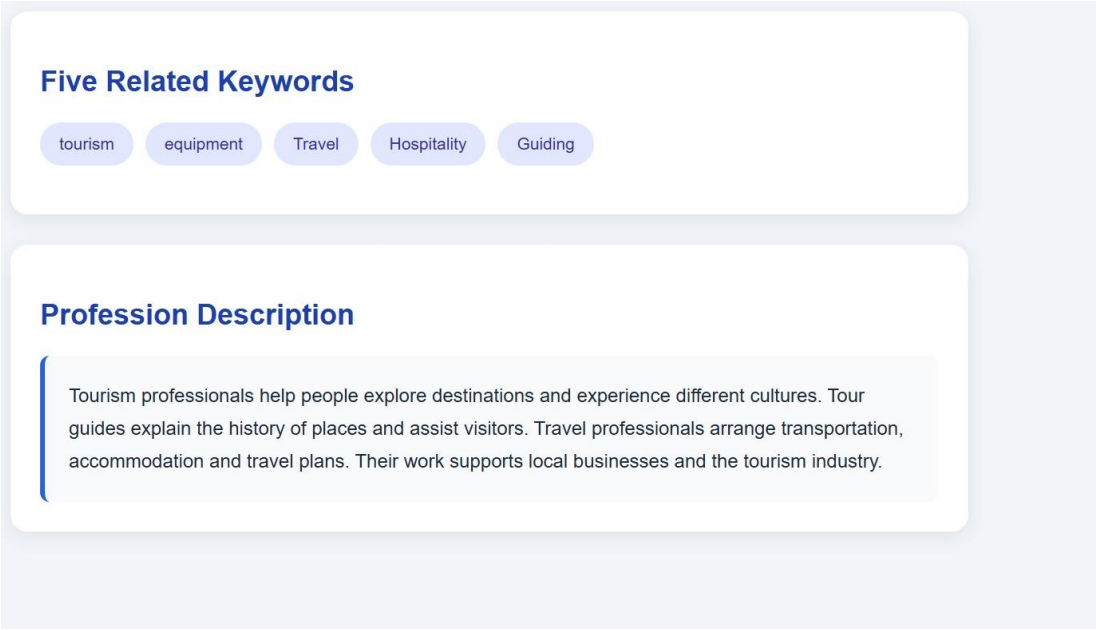
## 6.3 Classification Function

The classification function accepts an image path, loads the image, converts it to RGB, and invokes the CLIP processor. The model produces image-text similarity logits. Softmax normalization converts the logits into a relative score distribution. The index of the maximum score maps to the category list, and the corresponding score is returned for display

## 6.4 Application Screenshots and Interpretation

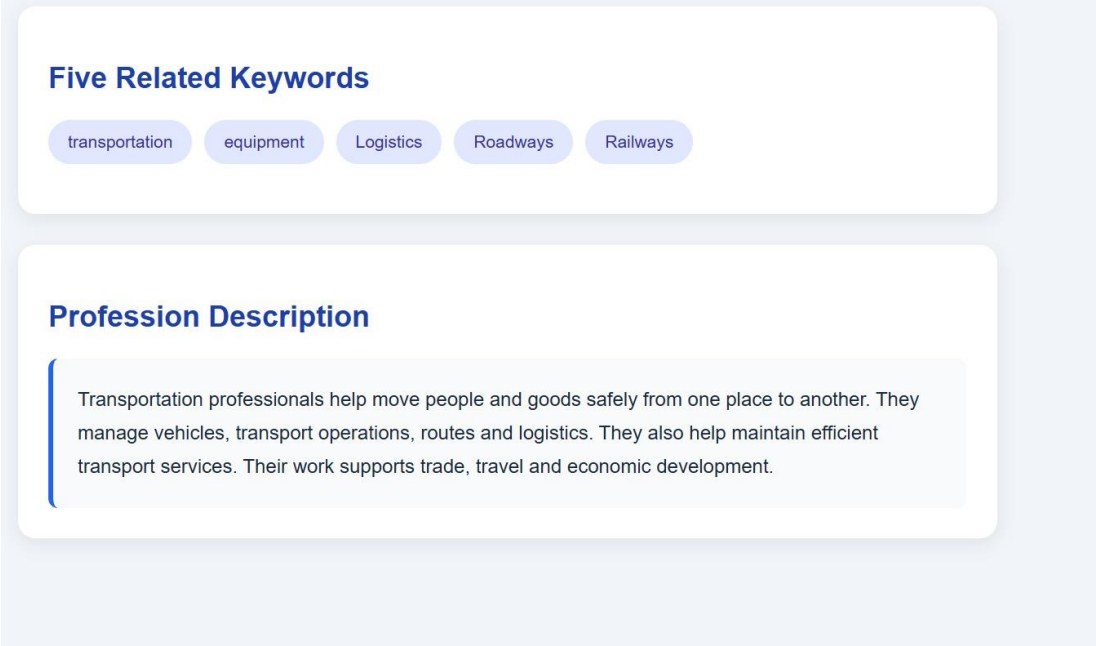
The following figures document the outputs obtained from the application. Each screenshot is followed by a brief interpretation. The confidence value is the softmax-normalized similarity score among the five candidate categories; it is not a calibrated probability of real-world correctness.

**Figure 6.4.1 – Tourism keywords and profession description**



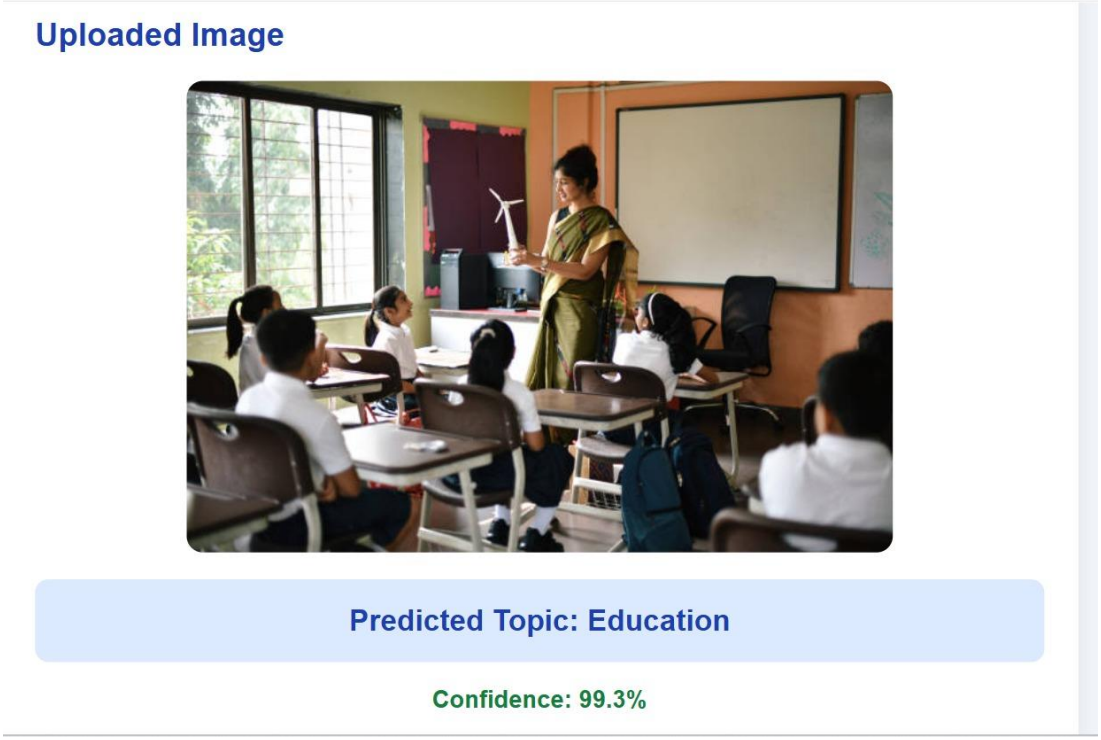
Meaning: The panel displays tourism-related terms such as tourism, travel, hospitality and guiding, followed by a short description of tourism professionals. It represents the text information shown after a Tourism classification.

**Figure 6.4.2 – Transportation keywords and profession description**




Meaning: The panel shows transportation-related terms including logistics, roadways and railways. The paragraph summarizes the role of transportation professionals in moving people and goods

**Figure 6.4.3 – Education image classification**



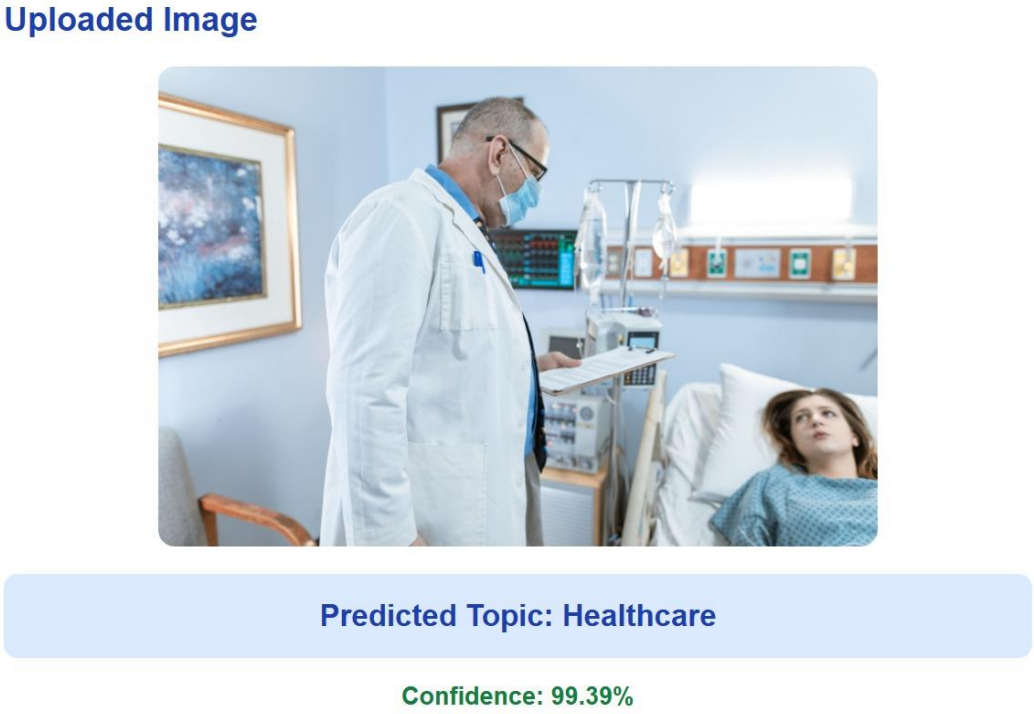

Meaning: The classroom image is classified as Education with a displayed score of 99.3%. The visible teacher, students and classroom setting align closely with the Education category

Figure 6.4.4 – Farming image classification



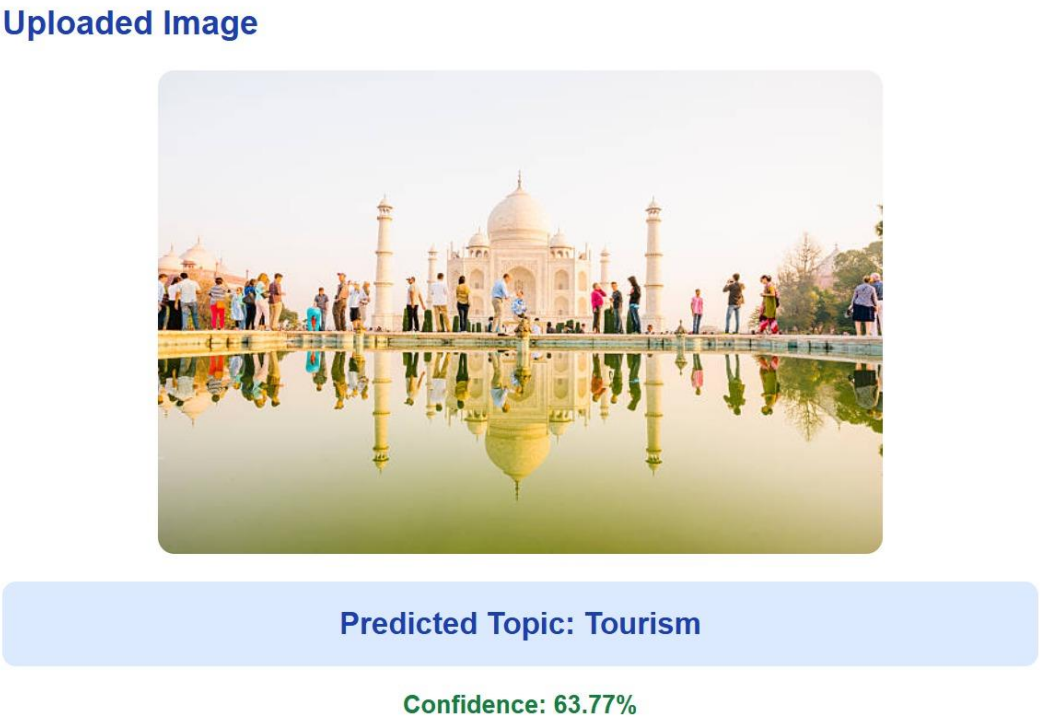

Meaning: The image of a person working in a field is classified as Farming with a score of 95.07%. The agricultural activity and cultivated field provide visual cues for this topic

Figure 6.4.5 – Healthcare image classification



Meaning: The hospital scene is classified as Healthcare with a score of 99.39%. The medical professional, patient and clinical environment are consistent with the Healthcare category

**Figure 6.4.6 – Tourism image classification**



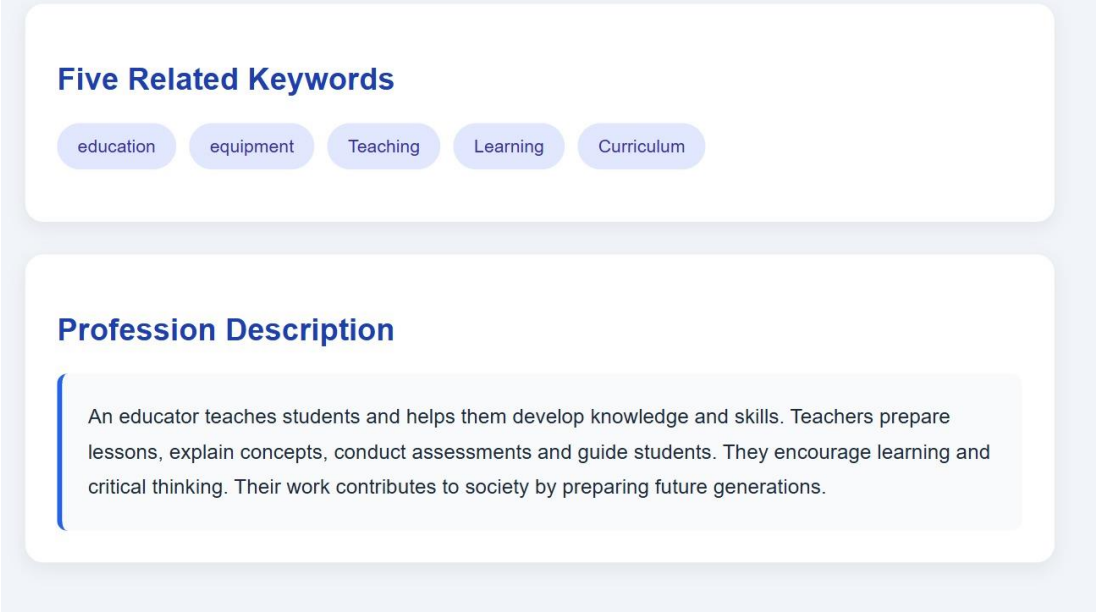

Meaning: The image of visitors at the Taj Mahal is classified as Tourism with a score of 63.77%. The comparatively lower score indicates weaker relative similarity than some other examples, although Tourism remains the top-ranked category

**Figure 6.4.7 – Transportation image classification**



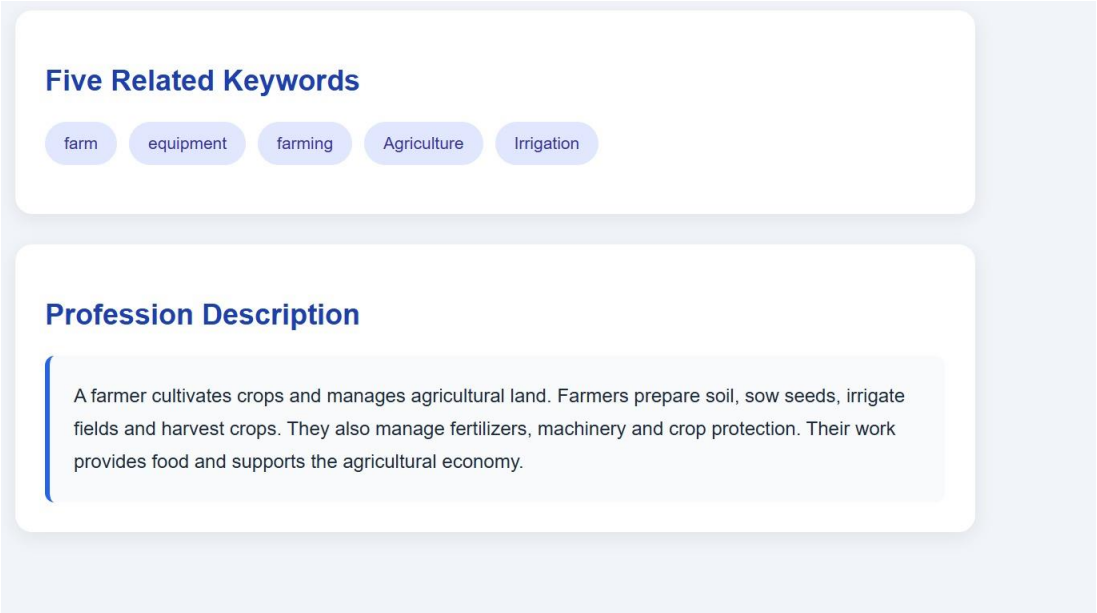
Meaning: The public bus image is classified as Transportation with a score of 84.87%. The bus and road context correspond to the Transportation category

**Figure 6.4.8 – Education keywords and profession description**



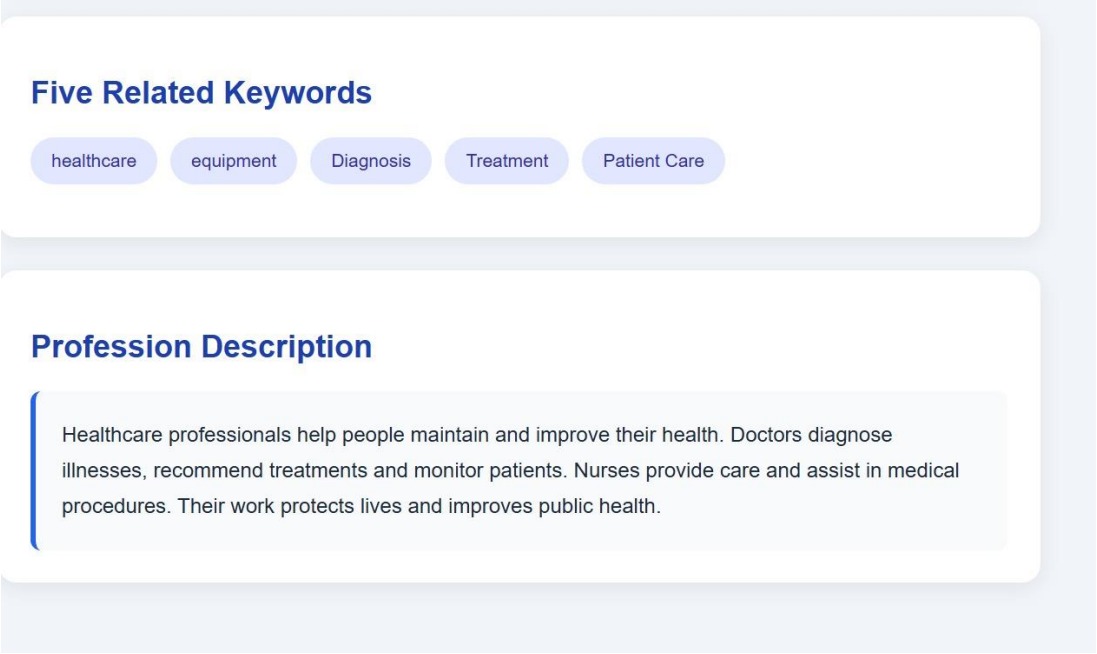

Meaning: The keywords include education, equipment, teaching, learning and curriculum. The accompanying paragraph explains the responsibilities of educators and the purpose of teaching

**Figure 6.4.9 – Farming keywords and profession description**




Meaning: The keywords include farm, equipment, farming, agriculture and irrigation. The description outlines soil preparation, sowing, irrigation and harvesting activities

Figure 6.4.10 – Healthcare keywords and profession description




Meaning: The panel lists healthcare, equipment, diagnosis, treatment and patient care. Its paragraph briefly describes how doctors and nurses support patients and public health

## 6.5 Keyword Generation Function

The keyword function constructs an instruction using the predicted category. The tokenizer prepares the prompt and FLAN-T5 generates a response. The decoded text is split into keyword candidates, normalized for whitespace and punctuation, and checked for duplicates. If the generated response contains fewer than five usable keywords, a category-specific fallback list can supply the remaining entries

## 6.6 Profession Description Mapping

A dictionary maps each supported category to a paragraph. For example, the healthcare entry explains that doctors diagnose illnesses, recommend treatments and monitor patients, while nurses provide care and assist with procedures. The farming entry describes soil preparation, sowing, irrigation and harvesting. Similar category-level descriptions are maintained for education, tourism and transportation

## 6.7 Flask Routes and Templates

The homepage route renders the upload template. The classification route handles the POST request, validates the file, saves it, calls the classification and text-generation functions, and passes the outputs to the result template. Templates avoid embedding large amounts of presentation markup inside Python code

## 6.8 User Workflow

10. Start the Flask server from the activated virtual environment.
11. Open http://127.0.0.1:5000 in a browser.
12. Select an image from the local computer.
13. Click the Classify Image button.
14. Wait for the model processing to complete.
15. Review the predicted topic and score.
16. Read the five related keywords and profession description.

### 6.9 Execution Commands

Activate the virtual environment: .\venv\Scripts\activate

Install dependencies: pip install torch transformers pillow flask sentencepiece

Run the server: python app.py

Open the local application: http://127.0.0.1:5000

# CHAPTER 7 – RESULTS AND SAMPLE OUTPUTS

This chapter presents the application outputs observed during testing. The screenshots show the topic selected by the model and the accompanying score, keywords and profession description

## 7.1 Summary of Demonstration Results

| Test image | Predicted topic | Displayed score |
|---|---|---|
| Classroom | Education | 99.3% |
| Agricultural field | Farming | 95.07% |
| Hospital scene | Healthcare | 99.39% |
| Taj Mahal / visitors | Tourism | 63.77% |
| City bus | Transportation | 84.87% |

These values are reproduced from the supplied application screenshots. They represent the normalized score shown by the application for the selected category and should not be interpreted as the model's measured accuracy

## 7.2 Education Result

The classroom image contains a teacher standing near the board and students seated at desks. CLIP ranks the education-related prompt highest. The output keywords include education, equipment, teaching, learning and curriculum. The profession description explains lesson preparation, concept explanation, assessment and guidance.

## 7.3 Farming Result

The farming image shows a person working in a cultivated field. The predicted category is Farming. Keywords include farm, equipment, farming, agriculture and irrigation. The description summarizes crop cultivation, soil preparation, sowing, irrigation, harvesting and machinery management

## 7.4 Healthcare Result

The healthcare image shows a medical professional beside a patient in a hospital setting. The predicted topic is Healthcare. Related keywords include healthcare, equipment, diagnosis, treatment and patient care. The profession paragraph refers to diagnosis, treatment, monitoring and nursing support

## 7.5 Tourism Result

The tourism image depicts visitors at a prominent destination. CLIP selects Tourism, with a displayed score of 63.77%. The relatively lower score among the sample outputs illustrates that the normalized score can vary according to image content and competition between candidate prompts. Keywords include tourism, equipment, travel, hospitality and guiding

## 7.6 Transportation Result

The transportation image shows a public bus on a road. The predicted category is Transportation and the displayed score is 84.87%. Keywords include transportation, equipment, logistics, roadways and railways. The description refers to moving people and goods, vehicles, routes, logistics and efficient transport services

## 7.7 Keyword and Profession Description Panels

The keyword panel presents five short terms as individual visual tags. The profession-description panel provides a paragraph in a bordered text area. These two panels enrich the category label: keywords provide quick scanning, while the paragraph supplies context. They are generated or selected after classification and do not influence the CLIP prediction itself

## 7.8 Additional Output Panels

The following panels complete the keyword and profession-description examples for the remaining categories.

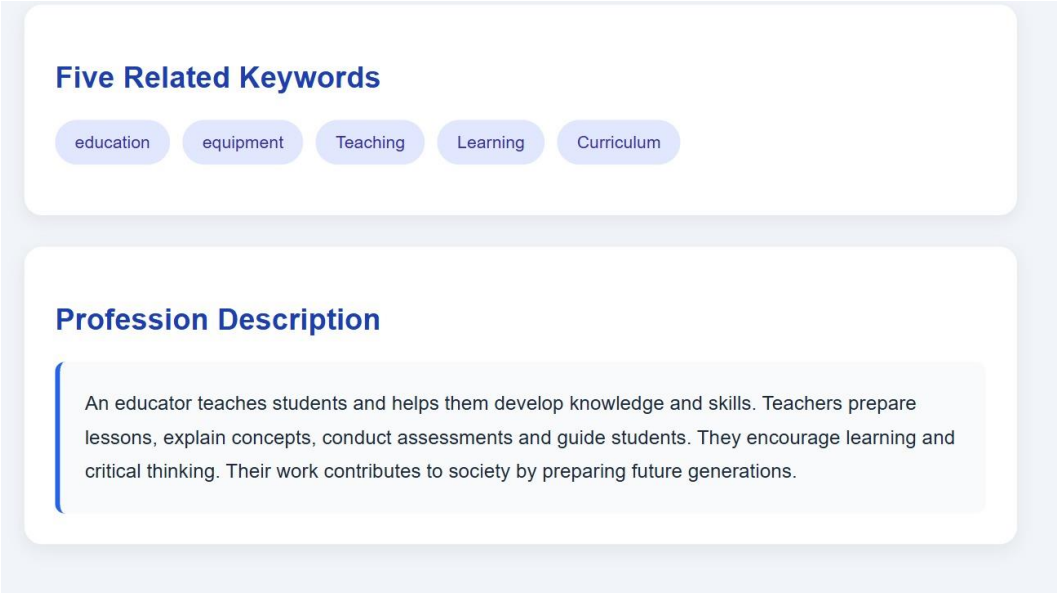



**Figure 7.8: Healthcare keywords and profession description.**

Meaning: The healthcare panel lists relevant terms such as diagnosis, treatment and patient care, followed by a concise explanation of healthcare professionals' responsibilities.

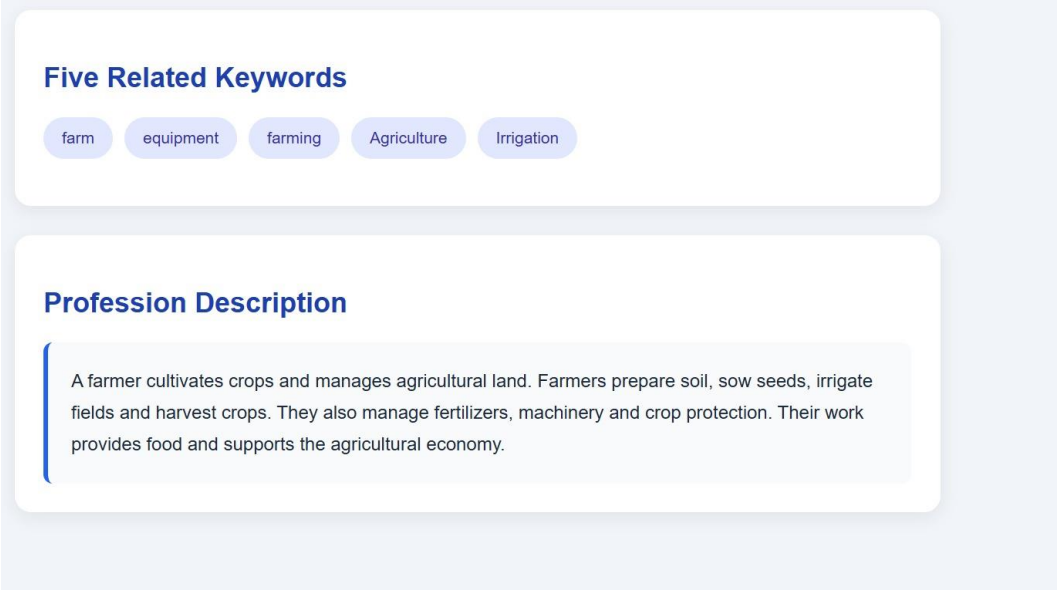



**Figure 7.9: Tourism keywords and profession description.**

Meaning: The tourism panel presents travel- and hospitality-related keywords and a short description of tourism professionals and visitor services.

## 7.9 Transportation Text Output

The transportation text-output panel is shown below.

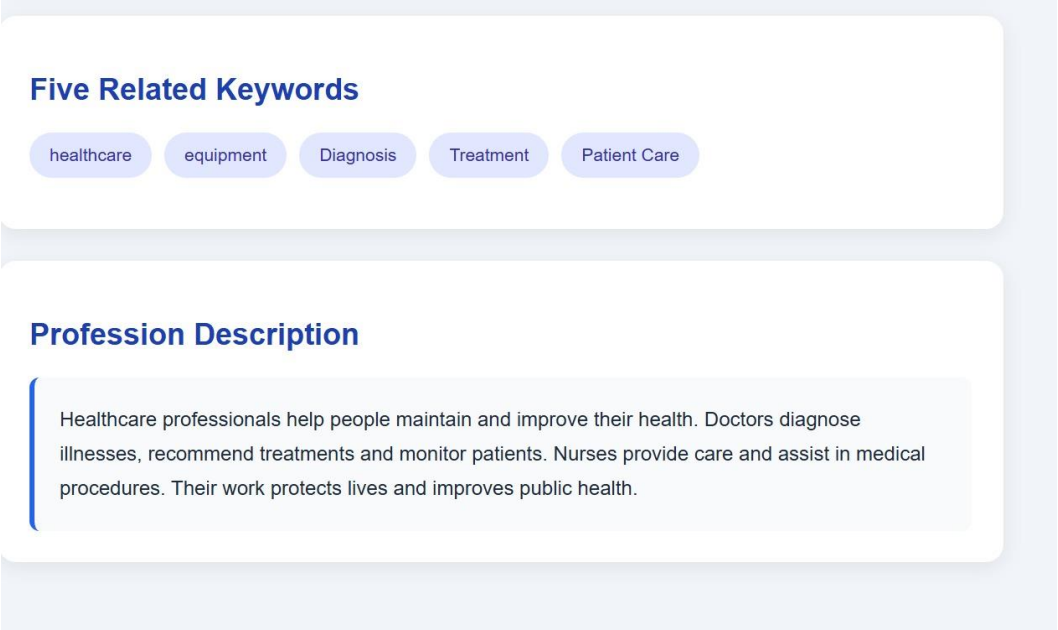



**Figure 7.10: Transportation keywords and profession description.**

Meaning: The panel displays transportation-related keywords and explains the work involved in moving people and goods through road, rail and logistics systems.

## 7.10 Consolidated Output Interpretation

Across the demonstration examples, the application completes the intended sequence: image upload, category prediction, keyword generation and description display. The screenshots establish that the interface is able to present outputs for all five supported topics. They do not, by themselves, establish generalization to unseen datasets or statistical accuracy

# CHAPTER 8 – TESTING, EVALUATION AND DISCUSSION

Testing verifies whether the implemented application behaves as expected under representative inputs. The tests below are functional checks; a formal model evaluation would require a labelled dataset and repeatable experimental procedure

## 8.1 Functional Test Cases

| Test ID | Input / action | Expected behaviour | Status |
|---|---|---|---|
| TC-01 | Open homepage | Upload interface is visible | Observed |
| TC-02 | Upload classroom image | Education output is displayed | Observed |
| TC-03 | Upload farming image | Farming output is displayed | Observed |
| TC-04 | Upload hospital image | Healthcare output is displayed | Observed |
| TC-05 | Upload destination image | Tourism output is displayed | Observed |
| TC-06 | Upload bus image | Transportation output is displayed | Observed |
| TC-07 | Inspect keyword panel | Five terms are shown | Observed |
| TC-08 | Inspect description panel | Category paragraph is shown | Observed |
| TC-09 | Submit without a file | Application should show validation feedback | Recommended |
| TC-10 | Upload unsupported file | Application should reject safely | Recommended |

## 8.2 Evaluation Metrics

A complete evaluation should measure both classification quality and application performance. The following metrics are appropriate:

| Metric | Definition / use |
|---|---|
| Accuracy | Correct predictions divided by total labelled examples |
| Precision | Correct positive predictions divided by all positive predictions for a class |
| Recall | Correct positive predictions divided by all actual examples of a class |
| F1-score | Harmonic mean of precision and recall |
| Confusion matrix | Shows class-wise prediction errors |
| Latency | Elapsed time from upload to result |
| Keyword validity | Whether output contains five distinct, relevant terms |
| Usability | Ease of upload and interpretation |

## 8.3 Accuracy versus Similarity Score

The interface displays a softmax-normalized CLIP score. A score such as 99.39% means that the selected candidate received a large share of the normalized similarity distribution for that particular input and candidate set. It does not mean that 99.39% of all images will be classified correctly. Accuracy must be measured over many labelled examples, including difficult and out-of-domain images

## 8.4 Discussion of Sample Scores

The classroom and hospital examples receive high displayed scores because their visual content strongly aligns with the corresponding category prompts. The tourism example receives a lower score, which may reflect competing visual interpretations, background content or prompt wording. A low relative score is a useful signal for caution, but a fixed threshold should not be adopted without validation

## 8.5 Reproducibility

For reproducible testing, the category prompts, model checkpoint, software versions, image files and inference settings should be recorded. The same input and configuration should be used when comparing prompt variants or alternative checkpoints. Results should be saved in a structured table rather than relying only on screenshots

# CHAPTER 9 – LIMITATIONS, SECURITY AND ETHICAL CONSIDERATIONS

A responsible AI application should communicate what it can and cannot do. This chapter documents technical limitations and practical safeguards relevant to image-upload systems

## 9.1 Model Limitations

- The classifier is restricted to five predefined categories.
- It selects the closest category even when an image is unrelated to all five.
- Images containing multiple activities may be ambiguous.
- Prompt wording can affect the ranking of candidate categories.
- Similarity scores are not calibrated correctness probabilities.
- Keyword generation may include generic, repeated or imperfect terms.
- Profession descriptions are predefined and may simplify real occupations.

## 9.2 Input and File Security

Uploaded files should be treated as untrusted input. The application should restrict permitted extensions, verify image content rather than relying only on a filename, impose a maximum file size and use secure filename handling. The upload directory should not permit execution of uploaded content. Temporary files should be removed according to a retention policy

## 9.3 Privacy

Images may contain faces, documents, locations or other personal information. A deployment should inform users about how images are processed, whether they are stored and when they are deleted. The prototype should avoid collecting unnecessary personal data and should not expose uploaded files to other users

## 9.4 Responsible Interpretation

The application is a demonstration tool and should not be used for consequential decisions. Healthcare-related images are classified only at a broad topic level; the application does not diagnose disease, recommend treatment or assess a patient's condition. Likewise, farming, education, tourism and transport descriptions are general informational summaries

## 9.5 Reliability and Error Handling

Potential failures include missing files, corrupted images, model download errors, insufficient memory and unexpected model output. Clear user-facing messages and server-side logging can help diagnose such failures. Error messages should avoid exposing internal paths, credentials or stack traces

## 9.6 Ethical Considerations

The model may reflect biases present in its pretraining data. Images from different regions, languages, lighting conditions or social contexts may be represented unevenly. Evaluation should include diverse examples, and users should be able to recognize that the predicted category is automated and may be wrong

# CHAPTER 10 – CONCLUSION AND FUTURE SCOPE

The final chapter summarizes the achievement of the project and identifies realistic extensions that can improve its capabilities

## 10.1 Conclusion

The AI Image Topic Classifier demonstrates the integration of computer vision and natural language processing through a web-based application. CLIP performs zero-shot image-to-topic classification for Farming, Education, Healthcare, Tourism and Transportation. FLAN-T5 generates five related keywords based on the predicted category, while a predefined mapping supplies a profession-related paragraph

Flask connects the model functions to the browser interface, and HTML/CSS provide a straightforward upload and result presentation. The supplied test screenshots show the application producing results for all five categories. The project provides practical experience in pretrained model use, image preprocessing, inference, text generation, backend integration and interface design

The project also emphasizes careful interpretation: the displayed score is a normalized similarity score rather than a verified accuracy percentage. A larger labelled evaluation set would be needed to establish classification performance beyond the sample demonstrations

## 10.2 Future Scope

### 10.2.1 Expansion of Categories

The candidate category list can be expanded to include sports, wildlife, technology, industry, entertainment, environment and public services. New categories should be tested for overlap and prompt ambiguity before being included

### 10.2.2 Improved Classification

Alternative vision-language checkpoints and prompt ensembles can be evaluated. A labelled dataset can be used to compare models using accuracy, precision, recall, F1-score and confusion matrices. An unknown or out-of-scope class can be introduced to reduce forced predictions

### 10.2.3 Dynamic Image Captioning

An image-captioning model can produce a sentence describing visible content rather than only assigning a topic. Captions could complement the category label and provide more specific information, subject to safeguards against hallucinated details

### 10.2.4 Richer Language Generation

FLAN-T5 can be prompted to generate explanations, summaries or structured information. A validation layer should check length, relevance and formatting. Profession descriptions may be generated dynamically in a future version, while the current predefined text remains a predictable baseline

### 10.2.5 Multilingual and Mobile Support

The interface and generated text can be extended to Hindi, Odia and other languages. A responsive mobile interface or dedicated mobile application could improve accessibility for users who primarily use smartphones

### 10.2.6 Deployment and Monitoring

The application can be deployed on a cloud platform with controlled access, HTTPS, resource monitoring and model caching. Monitoring can record processing latency and anonymized error statistics. Model updates should be tested before being introduced into the deployed system

# APPENDIX A – GLOSSARY

| Term | Explanation |
|---|---|
| Artificial Intelligence | Computational techniques that perform tasks associated with perception, reasoning or language. |
| Computer Vision | Methods that enable computers to process and interpret visual data. |
| Image Classification | Assigning an image to one or more predefined labels. |
| Vision-Language Model | A model trained to connect visual information with language. |
| CLIP | A model that aligns image and text representations. |
| Zero-Shot Learning | Performing a task using classes not used to train a task-specific classifier. |
| Embedding | A numerical representation of an input in a vector space. |
| Similarity Logit | A raw score measuring compatibility between image and text representations. |
| Softmax | A function that converts a vector of scores into normalized values. |
| FLAN-T5 | An instruction-tuned encoder-decoder language model. |
| Tokenizer | A component that converts text into model tokens. |
| Inference | Using a trained model to produce predictions or generated output. |
| Preprocessing | Preparing raw input in the format expected by a model. |
| RGB | A color representation using red, green and blue channels. |
| Flask | A lightweight Python web framework. |
| Template | A reusable HTML structure populated with application data. |
| Latency | Time taken to complete a processing operation. |
| Precision | A metric describing the correctness of positive predictions. |
| Recall | A metric describing the proportion of actual positives detected. |
| F1-score | A harmonic mean of precision and recall. |

# APPENDIX B – USER GUIDE

## B.1 Before Starting

Ensure that Python is installed and that the project dependencies are available. The first run may require an internet connection to download the pretrained model checkpoints. Adequate memory is recommended because both models are comparatively large

## B.2 Starting the Application

17. Open the project directory in Visual Studio Code.
18. Open a terminal in the project folder.
19. Activate the virtual environment.
20. Install dependencies if required.
21. Execute python app.py.
22. Wait until Flask reports that the local server is running.
23. Open the local URL in a browser.

## B.3 Classifying an Image

24. Click the file-selection button.
25. Choose a clear image related to one of the five categories.
26. Confirm the selected filename.
27. Click Classify Image.
28. Wait for inference and generation.
29. Read the predicted topic and score.
30. Review the keywords and profession paragraph.

## B.4 Tips for Better Demonstrations

- Use clear images with one dominant subject.
- Prefer images that visibly represent one of the supported categories.
- Avoid extremely small, blurry or corrupted files.
- Do not interpret the score as guaranteed correctness.
- Try examples from different settings to observe model behaviour.

## B.5 Troubleshooting

| **Issue** | **Possible action** |
|---|---|
| Server does not start | Check Python version, dependencies and terminal error |
| Model download fails | Check internet connectivity and Hugging Face access |
| Image is rejected | Use a supported image format and valid file |
| Slow first prediction | Allow model loading and checkpoint download to finish |
| Unexpected category | Try a clearer image; note the five-class restriction |
| Few keywords | Check generation output and fallback keyword logic |

# APPENDIX C – CATEGORY-WISE KEYWORD AND DESCRIPTION REFERENCE

## Education

Keywords: education, equipment, Teaching, Learning, Curriculum

Description: An educator teaches students and helps them develop knowledge and skills. Teachers prepare lessons, explain concepts, conduct assessments and guide students. They encourage learning and critical thinking. Their work contributes to society by preparing future generations.

## Farming

Keywords: farm, equipment, farming, Agriculture, Irrigation

Description: A farmer cultivates crops and manages agricultural land. Farmers prepare soil, sow seeds, irrigate fields and harvest crops. They also manage fertilizers, machinery and crop protection. Their work provides food and supports the agricultural economy.

## Healthcare

Keywords: healthcare, equipment, Diagnosis, Treatment, Patient Care

Description: Healthcare professionals help people maintain and improve their health. Doctors diagnose illnesses, recommend treatments and monitor patients. Nurses provide care and assist in medical procedures. Their work protects lives and improves public health.

## Tourism

Keywords: tourism, equipment, Travel, Hospitality, Guiding

Description: Tourism professionals help people explore destinations and experience different cultures. Tour guides explain the history of places and assist visitors. Travel professionals arrange transportation, accommodation and travel plans. Their work supports local businesses and the tourism industry.

## Transportation

Keywords: transportation, equipment, Logistics, Roadways, Railways

Description: Transportation professionals help move people and goods safely from one place to another. They manage vehicles, transport operations, routes and logistics. They also help maintain efficient transport services. Their work supports trade, travel and economic development.

# APPENDIX D – SAMPLE PROMPTS AND OUTPUT FORMATS

## E.1 Candidate Category Prompts

The following prompt examples illustrate a consistent way to represent the candidate classes. The exact prompts used by the implementation should be retained in the source code and documented for reproducibility.

| Category | Illustrative prompt |
|---|---|
| Farming | A photograph related to farming, agriculture or crop cultivation. |
| Education | A photograph related to education, teaching or a classroom. |
| Healthcare | A photograph related to healthcare, medical services or a hospital. |
| Tourism | A photograph related to tourism, travel or a tourist destination. |
| Transportation | A photograph related to transportation, vehicles or public transit. |

## E.2 Keyword Prompt

Illustrative instruction: “Generate five concise keywords related to the topic [predicted topic]. Return only the keywords separated by commas.”

## E.3 Result Data Structure

| Field | Example / meaning |
|---|---|
| image_path | Path used to display the uploaded image |
| topic | One of the five predicted categories |
| score | Normalized score for selected category |
| keywords | List containing five displayed keywords |
| description | Predefined profession-related paragraph |

## E.4 Output Presentation

The result page should present the category prominently, show the score with an explanatory label, and separate supporting keywords from the description. The design should avoid implying that the score is a guaranteed probability of correctness. A brief note can explain that the classifier chooses the closest match among the five available categories

# APPENDIX E – PROJECT CHECKLIST

| Item | Completion / review |
|---|---|
| Project environment configured | ☐ |
| Dependencies installed | ☐ |
| CLIP checkpoint loads | ☐ |
| FLAN-T5 checkpoint loads | ☐ |
| Image upload route works | ☐ |
| RGB conversion verified | ☐ |
| Five category prompts documented | ☐ |
| Softmax score interpretation documented | ☐ |
| Keyword parsing and fallback checked | ☐ |
| Profession descriptions verified | ☐ |
| All five category screenshots included | ☐ |
| Missing-file handling tested | ☐ |
| Unsupported file handling tested | ☐ |
| README completed | ☐ |
| GitHub URL inserted | ☐ |
| Report reviewed for formatting | ☐ |

## FINAL NOTE

This report documents the prototype as implemented and demonstrated. Before submission, verify the student details, guide information, actual repository URL, software versions and any additional institutional formatting requirements. Where a recommended safeguard or future feature has not been implemented, it should remain identified as a recommendation rather than a completed feature.

# SUPPLEMENTARY DISCUSSION 1 – IMAGE REPRESENTATION AND EMBEDDINGS

An image encoder converts visual input into a numerical representation called an embedding. An embedding is a vector that summarizes patterns learned by the model. It is not a literal list of objects or a human-readable description. The representation is useful because images with related semantic content can occupy nearby regions of the learned space.

In CLIP, the image embedding is compared with text embeddings generated by the text encoder. The model learns a shared representation space in which compatible image and text pairs have higher similarity. This mechanism allows the project to compare an uploaded classroom image with a sentence describing education, without training a separate classroom classifier.

Similarity is commonly calculated using a dot product between normalized vectors, which is equivalent to cosine similarity when both vectors have unit length. The model may scale the similarity values before returning logits. These logits are ranking scores, not direct percentages. Softmax converts the set of logits into a normalized distribution across the candidate prompts.

The interpretation depends on the candidate set. If the application offers only five categories, the model must select one of them even when the image represents something outside the scope. Adding an unknown category or applying a validated rejection threshold can reduce this forced-choice behaviour.

Embedding-based classification is therefore best understood as semantic matching. It is flexible and convenient for demonstrations, but prompt design, category overlap and domain differences can influence the output.

- Image embedding: compact numerical representation of visual content.
- Text embedding: numerical representation of a candidate description.
- Cosine similarity: measure of angular similarity between two vectors.
- Zero-shot prediction: ranking labels without task-specific classifier training.

# SUPPLEMENTARY DISCUSSION 2 – TRANSFORMER MODEL CONCEPTS

Transformer architectures use attention mechanisms to process sequences. Rather than relying only on sequential recurrence, attention allows a token to relate to other tokens in the input. This helps models capture context and dependencies across a sequence.

T5 uses an encoder-decoder structure. The encoder processes the input text and creates contextual representations. The decoder generates output tokens step by step while attending to the encoder representations and previously generated tokens. The same text-to-text framework can be applied to summarization, translation, question answering and instruction-following tasks.

FLAN-T5 is instruction-tuned using a mixture of tasks expressed as natural-language instructions. Instruction tuning improves the model's ability to respond to prompts that describe what should be done. In this project, the instruction is intentionally narrow: produce five keywords for a topic label.

Generated text is probabilistic and may vary with decoding settings. Greedy decoding selects the most likely next token at each step, while sampling introduces controlled randomness. For a short keyword list, deterministic decoding can make demonstrations more consistent. However, deterministic generation does not guarantee semantic correctness.

The application uses a lightweight post-processing step because language models do not always return exactly the requested format. Output parsing, duplicate removal and fallback terms make the final display more stable. These steps should be documented as application logic rather than attributed entirely to the language model.

- Encoder: transforms input tokens into contextual features.
- Decoder: generates output tokens conditioned on input and prior output.
- Instruction tuning: training that improves response to task instructions.
- Post-processing: formatting and validation applied after generation.

# SUPPLEMENTARY DISCUSSION 3 – WEB APPLICATION DESIGN

A web application is commonly divided into a client and a server. The browser acts as the client and presents the interface. The Flask server receives requests, performs processing and returns HTML. This arrangement allows the AI models to remain on the server side while the user interacts through a familiar browser.

An upload form generally uses the POST method because it sends file content to the server. The server checks the request, stores the file and starts the processing workflow. The response page is rendered from a template with variables containing the result. This is more maintainable than manually assembling a large HTML string in a Python route.

Templates support separation of concerns. Python controls application logic and supplies values, while HTML defines structure and CSS defines presentation. A consistent design system—heading sizes, spacing, cards and button styles—improves readability and reduces visual clutter.

During local development, the application is accessed through the loopback address 127.0.0.1. This address refers to the local computer and is not automatically accessible from the public internet. A production deployment requires a proper hosting environment, configuration, security controls and a production WSGI server.

Model inference can make a request slower than a conventional web route. Loading models once, using appropriate hardware, limiting image dimensions and avoiding repeated initialization can improve responsiveness. A progress indicator or clear wait message can help the user understand that processing is underway.

- Client: browser and user interface.
- Server: Flask routes and inference functions.
- Template: HTML populated with dynamic values.
- Localhost: local development endpoint.
- Production deployment: configured server environment with security and monitoring.

# SUPPLEMENTARY DISCUSSION 4 – DATA AND MODEL MANAGEMENT

Pretrained model checkpoints are external resources and should be identified by their repository names and versions. A project should record the model identifier, Transformers version and relevant inference settings. This information is important because model updates or library changes can affect output behaviour.

Model files may be large and are generally cached after the first download. A first run can therefore require more time and network access than later runs. The report should distinguish the time needed to load a checkpoint from the time needed to process a single image.

Uploaded images should be treated as temporary application data unless the project explicitly requires persistent storage. A clear directory policy prevents the upload folder from growing indefinitely. For a deployed application, automatic cleanup and access restrictions are advisable.

Dependency management improves reproducibility. A requirements file should list the packages needed to run the application. For a stable release, compatible versions may be pinned after testing. A virtual environment prevents dependency conflicts with unrelated Python projects.

Configuration values such as upload size, model identifiers and storage paths should ideally be kept in a configuration module or environment variables. This reduces the need to modify core logic when moving between development and deployment environments.

- Record model IDs and tested dependency versions.
- Cache model checkpoints rather than reloading for every request.
- Use a cleanup policy for temporary uploaded images.
- Maintain requirements.txt and a README.
- Avoid committing large model caches or private uploads.

# SUPPLEMENTARY DISCUSSION 5 – TESTING STRATEGY IN DETAIL

Testing should be performed at multiple levels. Unit tests verify individual functions such as image preprocessing, category mapping and keyword cleanup. Integration tests verify that Flask routes correctly call the model functions and pass the expected values to templates. End-to-end tests verify the complete upload-to-result workflow from the browser perspective.

Test images should cover typical examples, difficult examples and images outside the supported categories. Typical examples establish that the basic workflow functions. Difficult examples—such as low-light images, cluttered scenes or images with multiple subjects—help identify limitations. Out-of-scope examples reveal the forced-choice nature of the current classifier.

Testing should include negative cases. A missing file, empty filename, unsupported extension, corrupted image and oversized upload should not crash the application or expose internal information. The expected behaviour is a clear error message and a safe return to the upload interface.

Model evaluation requires ground-truth labels. Each test image should be labelled independently, and predictions should be recorded without changing the prompt after seeing the result. A confusion matrix can reveal whether Farming is frequently confused with Transportation or Tourism, for example. Class-wise precision and recall are more informative than a single overall score when classes are imbalanced.

Latency can be measured from request submission to result rendering. Multiple runs should be recorded because first-run model loading and later inference have different costs. The report should state the hardware and whether model loading is included in the measurement.

- Unit testing: individual functions.
- Integration testing: backend and model interaction.
- End-to-end testing: complete browser workflow.
- Negative testing: invalid and missing inputs.
- Dataset-based evaluation: labelled examples and metrics.

# SUPPLEMENTARY DISCUSSION 6 – USER EXPERIENCE AND ACCESSIBILITY

The interface should make the next action obvious. A prominent upload control and a clearly labelled classification button reduce ambiguity. The user should be able to see which file has been selected before submitting it. If processing takes time, the interface can display a status message to prevent repeated submissions.

Visual hierarchy helps users scan results. The predicted topic should be the most prominent element, while the score should be visually secondary and accompanied by an explanatory label. Keywords can be displayed as compact tags, and the profession description can be placed in a readable paragraph card.

Accessibility includes readable font sizes, sufficient contrast, keyboard-operable controls and meaningful labels. Color should not be the only way to communicate information. Images should have suitable alternative text when the interface is designed for screen-reader users. The layout should remain usable at different browser widths.

Responsive design allows the result page to adapt to laptop, tablet and mobile screens. Images should scale without distortion and text cards should not overflow. A consistent spacing system avoids both excessive empty space and cramped content.

User feedback is also part of usability. A successful result should clearly indicate completion, while errors should explain what the user can do next. Avoid exposing technical stack traces or internal file paths in the browser.

- Use explicit labels for controls.
- Keep output hierarchy consistent.
- Provide readable contrast and keyboard navigation.
- Scale images responsively.
- Show success, loading and error states.

# SUPPLEMENTARY DISCUSSION 7 – POSSIBLE ENHANCEMENTS ROADMAP

Enhancement work can be organized into phases so that the prototype remains manageable. The first phase should strengthen reliability and input validation. The second phase should improve evaluation through a labelled dataset. The third phase can add richer language outputs and more categories. Deployment and monitoring can be treated as a final phase.

Phase 1: validate file type and size, handle errors, clean temporary uploads and keep models loaded. These changes improve robustness without changing the central project idea.

Phase 2: collect a balanced sample of images for the five topics, define annotation guidelines and calculate standard classification metrics. Prompt alternatives can be compared on a fixed validation set to avoid selecting prompts based only on a few demonstrations.

Phase 3: integrate image captioning, dynamic descriptions, multilingual output or a category hierarchy. A hierarchical classifier could first identify a broad domain and then select a more specific subcategory. An unknown class could allow the system to abstain when no candidate is suitable.

Phase 4: deploy the application with HTTPS, access controls, resource limits, monitoring and a documented privacy policy. If the application is used by multiple users, storage isolation and deletion policies become important.

| Phase | Main deliverables |
|---|---|
| 1 – Reliability | Validation, error handling, cleanup |
| 2 – Evaluation | Labelled dataset, metrics, confusion matrix |
| 3 – Intelligence | Captioning, new categories, multilingual output |
| 4 – Deployment | Hosting, security, monitoring, privacy |

# SUPPLEMENTARY DISCUSSION 8 – LEARNING OUTCOMES AND PROJECT REFLECTION

The project provides an opportunity to connect concepts from machine learning, computer vision, natural language processing and software engineering. It shows how a pretrained model can be used through a library interface and how model inputs must be prepared in a specific format before inference.

An important learning outcome is understanding the difference between training and inference. Training changes model parameters using data and an optimization process. In this implementation, the pretrained checkpoints are used for inference; the project does not fine-tune CLIP or FLAN-T5 on a custom dataset.

The project also demonstrates the importance of integration. A model by itself is not a complete user-facing product. File handling, validation, routing, templates, output formatting and error messages are necessary to make the functionality accessible and dependable.

Another learning outcome is critical interpretation of AI output. A high normalized score can appear impressive but does not establish correctness across a population of images. A formal evaluation must use independent labelled examples, appropriate metrics and documented experimental settings.

Finally, the project illustrates iterative development: define a narrow scope, implement the core workflow, test representative inputs, document limitations and propose future improvements. This process is applicable to many AI-enabled engineering applications.

- Understand vision-language similarity and zero-shot classification.
- Use pretrained Transformer models through Python libraries.
- Integrate inference functions into a Flask web application.
- Design and test a file-upload workflow.
- Interpret model scores responsibly and identify future work.